



Fachhochschul-Bachelorstudiengang
SOFTWARE ENGINEERING
A-4232 Hagenberg, Austria

Digitale Signalverarbeitung mit FAUST

Bachelorarbeit

zur Erlangung des akademischen Grades
Bachelor of Science in Engineering

Eingereicht von

Xianglei Wu

Begutachtet von FH-Prof. DI (FH) Dr. Erik Pitzer

Hagenberg, 05.08 2026

Erklärung

Ich erkläre eidesstattlich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen nicht benutzt und die den benutzten Quellen entnommenen Stellen als solche gekennzeichnet habe. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt.

Die vorliegende, gedruckte Bachelorarbeit ist identisch zu dem elektronisch übermittelten Textdokument.

05.08.2026
Datum

Unterschrift

Inhaltsverzeichnis

Abstract	v
Kurzfassung	vi
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Forschungsfrage und Zielsetzung	2
1.3 Aufbau der Arbeit	3
2 Grundlagen	5
2.1 Digitale Audiosignale und blockweise Verarbeitung	5
2.1.1 Abtastung und diskrete Sample-Folgen	5
2.1.2 Audio-Blöcke und Echtzeitdeadline	6
2.2 Elementare DSP-Bausteine	7
2.2.1 Verstärkungsstufe	7
2.2.2 Rekursionsfilter (IIR-Filter) und Biquad-Struktur	8
2.2.3 FIR-Filter und Faltung	9
2.3 Frequenzanalyse mit diskreter Fourier-Transformation (DFT) und Fast-Fourier-Transformation (FFT)	10
2.3.1 Zeitbereich, Frequenzbereich und DFT	10
2.3.2 Fast-Fourier-Transformation	10
2.4 Performanzrelevante Implementierungsaspekte	11
2.4.1 Compileroptimierung und automatische Vektorisierung	11
2.4.2 Messgrößen	12
2.5 Zusammenfassung	12
3 Faust: Functional Audio Stream	13
3.1 Einführung	13
3.2 Programmmodell	14
3.3 Syntax: Blockdiagramm-Komposition	14
3.4 Semantik	15
3.5 Compiler-Architektur	15
3.6 Codegenerierung	16
3.6.1 Skalarer Modus	17
3.6.2 Vektor-Modus	17
3.6.3 Parallel-Modus	17

3.6.4	Generierte Code	18
3.7	Backends und Architekturdateien	18
3.8	Performance	18
3.9	Zusammenfassung	19
4	Versuchsdesign und Benchmark-Methodik	20
4.1	Definition der untersuchten DSP-Bausteine	20
4.2	Benchmark-Tools und Messgrößen	20
4.3	Hardwareumgebung	20
4.4	Compilerkonfigurationen	20
4.5	Testdatengenerierung	20
5	Implementierung der Benchmarks	21
5.1	Umsetzung der FAUST- und C++-Code	21
5.2	Gemeinsame Test-Schnittstelle	21
5.3	Build-Systeme	21
5.4	Sicherstellung einer fairen Vergleichsbasis	21
5.5	Verifikation der Ergebnisse	21
6	Evaluation der Messergebnisse	22
6.1	Mikrobenchmark-Ergebnisse	22
6.2	Einfluss von Compileroptimierungen und Codegenerierung	22
6.3	Numerische Genauigkeit	22
6.4	Codelänge und Implementierungskomplexität	22
6.5	Zusammenfassung der Messergebnisse	22
7	Diskussion	23
7.1	Interpretation der Ergebnisse im Hinblick auf die Forschungsfrage	23
7.2	Limitationen des Versuchsaufbaus	23
7.3	Bedrohung der Validität	23
8	Zusammenfassung und Ausblick	24
8.1	Zusammenfassung der zentralen Ergebnisse	24
8.2	Ausblick auf künftige Forschungsarbeiten	24
Quellenverzeichnis		25
	Literatur	25
	Online-Quellen	25

Abstract

This should be a 1-page (maximum) summary of your work in English.

Kurzfassung

An dieser Stelle steht eine Zusammenfassung der Arbeit, Umfang max. 1 Seite. ...

Kapitel 1

Einleitung

1.1 Motivation und Problemstellung

Die digitale Signalverarbeitung (Digital Signal Processing, DSP) ist ein wichtiges Anwendungsfeld in der Informatik besonderen Wert. Ifeachor und Jervis (2002) nennen unter anderem Anwendungen in der Telekommunikation, Audiotechnik, Sprachverarbeitung und Medizintechnik. In diesen Anwendungsbereichen sind Laufzeiteffizienz, deterministische Latenz und numerische Präzision von besonderer Bedeutung.

DSP-Algorithmen werden traditionell in C oder C++ implementiert, unter anderem wegen des direkten Hardwarezugriffs und der hochoptimierten Codeerzeugung (Levine & Skolnick, 1997).

Die manuelle Übertragung mathematischer Modelle in ausführbaren Code ist jedoch aufwendig und erschwert die Portierung auf unterschiedliche Zielplattformen. Daraus kristallisierten sich in den letzten zwei Jahrzehnten domänenspezifische Sprachen (Domain-Specific Languages, DSLs) für die Audio-Signalverarbeitung, welches durch Abstraktion und Code-Generierung die Probleme bewältigen soll. (Orlarey et al., 2009, S. 2–6)

Orlarey et al. (2009) beschreiben die domänenspezifische Sprache FAUST (Functional Audio Stream), entwickelt am GRAME in Lyon, als eine funktionale, blockbasierte Programmiersprache, die speziell für Audio-DSP entworfen wurde. Programme werden als algebraische Komposition von Signalverarbeitungsoperatoren formuliert. Die aktuelle FAUST-Dokumentation beschreibt die Codegenerierung für mehrere Zielsprachen und Zielplattformen, darunter C++, Rust, JavaScript und WebAssembly (GRAME - Centre National de Création Musicale, 2025). Sie dokumentiert zudem Architekturdateien, mit denen aus einer FAUST-Quellbeschreibung unter anderem Audio-Plug-ins, Anwendungen für eingebettete Systeme und Webanwendungen erzeugt werden können (GRAME - Centre National de Création Musicale, 2025).

Die praktische Leistungsfähigkeit dieses Ansatzes wird am Beispiel des FAUST-STK untersucht. FAUST-STK ist eine Sammlung virtueller Musikinstrumente, deren Modelle überwiegend auf Wellenleitersynthese und teilweise auf modaler Synthese beruhen (Michon & Smith, 2011, S. 1). Die aus dem Synthesis ToolKit und SynthBuilder übernommenen Modelle wurden für die FAUST-Semantik angepasst und hinsichtlich ihrer Effizienz optimiert (Michon & Smith, 2011, S. 5–6).

Die Quelltextgrößenanalyse zeigt, dass die FAUST-Implementierungen bei den untersuchten Modellen kompakter sind als die entsprechenden STK-C++-Implementierungen. Der ausgewiesene Größenvorteil beträgt je nach Modell zwischen 22,91 Prozent und 80,20 Prozent (Michon & Smith, 2011, Tab. 1, S. 6). Zusätzlich vergleicht die Studie die CPU-Last von Pure-Data-Plug-ins, die auf STK-C++-Code beziehungsweise FAUST-generiertem Code basieren (Michon & Smith, 2011, Tab. 2, S. 6).

Für die praktische Eignung muss der durch FAUST generierte Code sowohl funktional korrekt als auch laufzeiteffizient sein. Scaringella et al. (2003) thematisieren die automatische Vektorisierung, während Letz et al. (2010) Möglichkeiten der Parallelisierung im FAUST-Umfeld behandeln. Die FAUST-Dokumentation verweist zudem auf Optimierungen, die den vollständigen Signalfluss berücksichtigen (Orlarey et al., 2009). Diese Eigenschaften lassen jedoch keine allgemeine Aussage darüber zu, ob FAUST-generierter C++-Code einer manuell implementierten C++-Referenz bei konkreten DSP-Bausteinen überlegen, gleichwertig oder unterlegen ist.

Direkte Vergleiche liegen für zeitbereichsbasierte Algorithmen vor. Für den Reverb-Algorithmus *Freeverb* berichten Orlarey et al. (2009) Benchmarkergebnisse, deren Ausprägung von Compiler und Codegenerierungsstrategie abhängt. Die in dieser Arbeit berücksichtigte Vergleichsliteratur untersucht überwiegend zeitbereichsbasierte Algorithmen wie Reverbs, Delays und physikalische Modelle. Ein Vergleich FFT-bezogener Bausteine wird daher ergänzend untersucht. Für einen fairen Vergleich folgen beide Implementierungen demselben algorithmischen Ablauf und werden über eine gemeinsame Schnittstelle in die Benchmark-Infrastruktur eingebunden. Auf explizit programmierte Single-Instruction-Multiple-Data-Intrinsics (SIMD-Intrinsics) wird verzichtet. Die automatische Vektorisierung des Compilers bleibt für beide Varianten aktiv.

In der vorliegende Arbeit wird untersucht, wie sich FAUST-generierter C++-Code gegenüber einer manuell implementierten C++-Referenz bei elementaren DSP-Bausteinen verhält. In Betracht gezogen Bausteine: eine Verstärkungsstufe (Gain-Stufe), ein Biquad-Filter, Finite-Impulse-Response-Filter (FIR-Filter) unterschiedlicher Länge sowie FFTs unterschiedlicher Größe. Der Schwerpunkt liegt auf reproduzierbaren Mikrobenchmarks, nicht auf einer vollständigen Audioanwendung.

1.2 Forschungsfrage und Zielsetzung

In der vorliegende Arbeit wird untersucht, wie sich FAUST-generierter C++-Code gegenüber einer manuell implementierten C++-Referenz bei elementaren DSP-Bausteinen verhält. In Betracht gezogen Bausteine: eine Verstärkungsstufe (Gain-Stufe), ein Biquad-Filter, Finite-Impulse-Response-Filter (FIR-Filter) unterschiedlicher Länge sowie FFTs unterschiedlicher Größe. Der Schwerpunkt liegt auf reproduzierbaren Mikrobenchmarks, nicht auf einer vollständigen Audioanwendung. Reproduzierbaren Vergleiche mit unterschiedlichen Datenabhängigkeiten -und Skalierungseigenschaften sind die Anforderungen. Neben der Laufzeit werden Speicherverbrauch und Binärgröße betrachtet. Ergänzend werden Codelänge und Implementierungskomplexität analysiert.

Die übergeordnete Forschungsfrage lautet:

Wie unterscheidet sich FAUST-generierter C++-Code von einer manuell implementierten C++-Referenz bei ausgewählten DSP-Bausteinen und FFTs

hinsichtlich Laufzeit, Speicherverbrauch, Binärgröße, Codelänge und Implementierungskomplexität?

Zur Beantwortung dieser Forschungsfrage werden folgende Teilfragen untersucht:

- Wie groß ist der Performanzunterschied hinsichtlich der CPU-Zeit pro Audio-Block zwischen FAUST-generiertem und manuell implementiertem C++-Code bei Gain-Stufen, Biquad-Filtern, FIR-Filtern und FFTs?
- Wie verändert sich der Laufzeitunterschied zwischen beiden Implementierungen in Abhängigkeit von Blockgröße, FIR-Filterlänge und FFT-Größe?
- Wie unterscheiden sich FAUST-generierter und manuell implementierter C++-Code hinsichtlich des Speicherverbrauchs während der Ausführung der untersuchten DSP-Bausteine?
- Welche Unterschiede bestehen zwischen FAUST-generiertem und manuell implementiertem C++-Code hinsichtlich der Binärgröße der erzeugten Programme?
- Welchen Einfluss haben Compileroptimierungen und automatische Vektorisierung auf beide Implementierungen?
- Welche Unterschiede bestehen hinsichtlich Codelänge und Implementierungskomplexität der beiden Ansätze?

1.3 Aufbau der Arbeit

Die Bachelorarbeit ist in acht Kapitel gegliedert. Sie umfasst die theoretischen Grundlagen, die Beschreibung von FAUST, das Versuchsdesign, die Implementierung der Benchmark-Kerne, die Evaluation sowie die Einordnung der Ergebnisse.

Nach dieser Einleitung vermittelt Kapitel 2 die für die Arbeit relevanten Grundlagen der digitalen Signalverarbeitung. Dazu gehören Audio-DSP, Verstärkungsstufen, rekursive Filter und FIR-Filter, die Fast-Fourier-Transformation sowie Anforderungen an Echtzeitverarbeitung.

Kapitel 3 stellt die domänenspezifische Sprache FAUST vor. Es erläutert die funktionale Syntax und Semantik, die zugrunde liegende Compiler-Architektur sowie die für die Arbeit relevanten Codegenerierungs-Backends.

Kapitel 4 beschreibt das Versuchsdesign und die Benchmark-Methodik. Es definiert die untersuchten DSP-Bausteine, die Messgrößen, die Zielplattformen, die Compilerkonfigurationen und die Maßnahmen zur Sicherung reproduzierbarer Messungen.

Kapitel 5 dokumentiert die praktische Umsetzung der FAUST- und C++-Varianten. Neben den DSP-Bausteinen werden die gemeinsame Schnittstelle, das Build-System und die Maßnahmen zur Gewährleistung einer fairen Vergleichsbasis beschrieben.

Kapitel 6 enthält die Evaluation. Die Ergebnisse der Mikrobenchmarks werden hinsichtlich Laufzeit, Speicherverbrauch und Binärgröße ausgewertet. Zusätzlich werden Skalierungseffekte durch unterschiedliche Blockgrößen, Filterlängen und FFT-Größen sowie der Einfluss von Compileroptimierungen analysiert. Im Anschluss wird die Erfahrung mit der Handhabung von FAUST bewertet.

Kapitel 7 diskutiert die Ergebnisse im Hinblick auf die Forschungsfrage. Dabei werden Limitationen des Versuchsaufbaus, Bedrohungen der Validität und sowie die Codelänge und Implementierungskomplexität analysiert.

Kapitel 8 fasst die zentralen Ergebnisse zusammen und gibt einen Ausblick auf mögliche Erweiterungen, etwa die Untersuchung vollständiger DSP-Pipelines oder weiterer Zielplattformen.

Kapitel 2

Grundlagen

Dieses Kapitel vermittelt die signaltheoretischen und laufzeitbezogenen Grundlagen für den Vergleich von Faust-generiertem und manuell implementiertem C++-Code. Im Mittelpunkt stehen digitale Audiosignale, eine Verstärkungsstufe (Gain-Stufe), Biquad- und Finite-Impulse-Response-(FIR)-Filter, die Fast-Fourier-Transformation (FFT) sowie die Anforderungen blockbasierter Echtzeitverarbeitung.

2.1 Digitale Audiosignale und blockweise Verarbeitung

2.1.1 Abtastung und diskrete Sample-Folgen

Ein analoges Audiosignal liegt zeitkontinuierlich vor. Für die digitale Verarbeitung wird es in regelmäßigen Zeitabständen abgetastet und als diskrete Folge von Sample-Werten dargestellt. Jedes Sample besitzt einen festen zeitlichen Abstand zum vorherigen Sample (Rossing, 2007, Kap. „Acoustic Signal Processing“, S. 520–522). Die Abtastfrequenz f_s bestimmt die Zahl der Messungen pro Sekunde; das Abtastintervall beträgt

$$T_s = \frac{1}{f_s}. \quad (2.1)$$

Die Beziehung zwischen Abtastfrequenz und Abtastintervall folgt aus der Definition der Abtastfrequenz für digitale Audiosignale (Rossing, 2007, S. 520–522). Die digitale Darstellung eines kontinuierlichen Signals $x(t)$ lautet damit

$$x[n] = x(nT_s), \quad n \in \mathbb{Z}. \quad (2.2)$$

Diese zeitdiskrete Darstellung beschreibt die Überführung eines kontinuierlichen Signals in eine Sample-Folge (Rossing, 2007, S. 520–522). Das Nyquist-Shannon-Kriterium fordert, dass die Abtastfrequenz mindestens doppelt so hoch wie die höchste im Signal enthaltene Frequenz sein muss. Andernfalls können Frequenzanteile nicht eindeutig rekonstruiert werden (Tan & Jiang, 2018, Kap. 2.1, S. 19–26).

$$f_s \geq 2f_{\max}. \quad (2.3)$$

Diese Bedingung bildet die Grundlage einer aliasfreien digitalen Darstellung (Rossing, 2007, S. 521–522). In der vorliegenden Arbeit werden Audiosignale als Folgen von Gleit-

kommawerten verarbeitet. Die Quantisierung ist daher nur als Übergang von der analogen in die digitale Darstellung relevant. Eine separate Untersuchung der Bittiefe oder der Quantisierungsqualität ist nicht Gegenstand der Benchmarks.

2.1.2 Audio-Blöcke und Echtzeitdeadline

Audiosysteme zur digitalen Signalverarbeitung (DSP) können Eingangssamples sowohl samplebasiert als auch blockbasiert verarbeiten. Blockbasierte Verfahren werden insbesondere bei der digitalen Filterung, Faltung, FFT und anderen echtzeitfähigen DSP-Anwendungen eingesetzt (Tan & Jiang, 2018, S. 727). Ein Block besteht aus N aufeinanderfolgenden Samples. Bei einer Abtastfrequenz f_s und einer Abtastperiode $T_s = 1/f_s$ besitzt ein Block die zeitliche Dauer

$$T_{\text{Block}} = NT_s = \frac{N}{f_s}. \quad (2.4)$$

Die Blockdauer ergibt sich aus der Anzahl der Samples und dem zeitlichen Abstand zwischen zwei aufeinanderfolgenden Samples. Bei der überlappenden Blockverarbeitung werden die Eingangsdaten in zeitlich versetzten Analysefenstern verarbeitet, wobei aufeinanderfolgende Blöcke mit N Samples einen gemeinsamen Bereich enthalten (Tan & Jiang, 2018, S. 499). Bei einer Überlappung von 50 Prozent besteht der zweite Block aus der zweiten Hälfte des ersten Blocks und einer neuen zweiten Hälfte (Tan & Jiang, 2018, S. 499). Die Rekonstruktion der überlappenden Ausgangsblöcke erfolgt anschließend durch Overlap-Add (Tan & Jiang, 2018, S. 500). Für eine Echtzeitverarbeitung muss die Berechnung rechtzeitig vor dem Eintreffen beziehungsweise der Ausgabe des nächsten Datenabschnitts abgeschlossen sein. Für eine samplebasierte Echtzeitverarbeitung müssen dazu die Analog-Digital-Umwandlung (ADC), die DSP-Verarbeitung und die Ausgabe innerhalb des zeitlichen Intervalls bis zum nächsten Ausgabezeitpunkt abgeschlossen werden (Tan & Jiang, 2018, S. 761). Für eine blockbasierte Verarbeitung lässt sich diese Echtzeitbedingung durch

$$T_{\text{exec}} \leq T_{\text{Deadline}} \quad (2.5)$$

ausdrücken. Dabei bezeichnet T_{exec} die tatsächliche Ausführungszeit des Algorithmus. Die Deadline hängt von der Organisation der Blockverarbeitung ab. Bei nicht überlappenden Blöcken entspricht die verfügbare Zeit der Blockdauer:

$$T_{\text{Deadline}} = T_{\text{Block}} = \frac{N}{f_s}. \quad (2.6)$$

Bei überlappender Verarbeitung wird dagegen alle H Samples ein neuer Verarbeitungszyklus ausgelöst. Die zeitliche Frist zwischen zwei aufeinanderfolgenden Zyklen entspricht daher der Hop-Zeit:

$$T_{\text{Deadline}} = T_{\text{Hop}} = \frac{H}{f_s}. \quad (2.7)$$

Die Hop-Zeit stellt damit die maßgebliche Rechenzeitfrist für eine überlappende blockbasierte Echtzeitverarbeitung dar (Tan & Jiang, 2018, S. 499–500, 761). Die Blockgröße und die Hop-Größe beeinflussen unterschiedliche Eigenschaften des Systems. Eine kleinere Blockgröße reduziert die durch die Pufferung verursachte Latenz, verkürzt jedoch

zugleich die verfügbare Zeit für die Verarbeitung. Bei einer Überlappung von 50 Prozent gilt beispielsweise

$$H = \frac{N}{2}. \quad (2.8)$$

Die Blockgröße und die Hop-Größe sind deshalb zentrale Parameter für die Konfiguration und Bewertung von Echtzeit-Benchmarks.

2.2 Elementare DSP-Bausteine

2.2.1 Verstärkungsstufe

Eine Verstärkungsstufe skaliert jedes Eingangssample mit einem konstanten Verstärkungsfaktor g . Für ein Eingangssignal $x[n]$ ergibt sich das Ausgangssignal $y[n]$ zu

$$y[n] = g \cdot x[n]. \quad (2.9)$$

Diese Skalierung stellt eine elementare lineare Operation der digitalen Audioverarbeitung dar (Smith, 2007). Der Baustein benötigt pro Sample lediglich eine Multiplikation und dient deshalb als einfacher Referenzfall für den Vergleich des durch Faust erzeugten Codes mit der C++-Referenz. Bei einer Angabe der Verstärkung in Dezibel gilt

$$g = 10^{G_{\text{dB}}/20}. \quad (2.10)$$

Diese Umrechnung folgt aus der Definition eines Amplitudenverhältnisses in Dezibel (Smith, 2007). Abbildung 2.1 veranschaulicht eine Verstärkungsstufe mit $g = 0.5$. Das Ausgangssignal besitzt für jedes Sample die halbe Amplitude des Eingangssignals, was einer Absenkung von ungefähr -6.02 dB entspricht. Die Verstärkungsstufe bildet

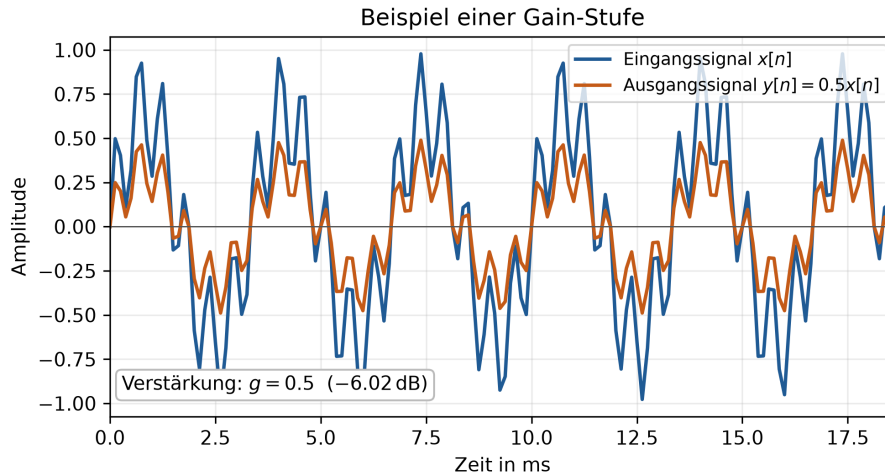


Abbildung 2.1: Beispiel einer Verstärkungsstufe mit $g = 0.5$. Die Amplitude jedes Eingangssamples wird halbiert. Eigene Darstellung nach (Smith, 2007).

einen einfachen, vollständig datenparallelen Benchmark-Kern. Vektorisierung bezeichnet dabei die Ausführung derselben Operation auf mehreren Datenwerten durch Single-Instruction-Multiple-Data-Instruktionen (SIMD-Instruktionen). Eine sampleweise Skalierung besitzt daher ein hohes Vektorisierungspotenzial (Scaringella et al., 2003).

2.2.2 Rekursionsfilter (IIR-Filter) und Biquad-Struktur

Ein Rekursionsfilter, auch Infinite-Impulse-Response-Filter (IIR-Filter) genannt, verwendet neben aktuellen und vergangenen Eingangswerten auch vergangene Ausgangswerte. Die Rückkopplung führt dazu, dass der Filterzustand über mehrere Samples erhalten bleibt. Für die Untersuchung wird ein Biquad-Filter, also ein IIR-Filter zweiter Ordnung, verwendet. Seine allgemeine Differenzengleichung lautet bei normiertem $a_0 = 1$

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2]. \quad (2.11)$$

IIR-Filter lassen sich durch solche Differenzengleichungen beschreiben. Die Koeffizienten b_i bilden den Vorwärtzweig, während die Koeffizienten a_i den Rückkopplungszweig bestimmen (Smith, 2007). Abbildung 2.2 zeigt die Wirkung eines Biquad-IIR-Tiefpasses zweiter Ordnung mit einer Grenzfrequenz von 800 Hz. Wie beim FIR-Beispiel enthält das synthetische Eingangssignal Anteile bei 300 Hz und 1800 Hz. Der Frequenzgang und das Ausgangssignal veranschaulichen die Dämpfung hoher Frequenzanteile durch die rekursive Filterstruktur. Eine IIR-Implementierung muss die vergangenen Ein- und

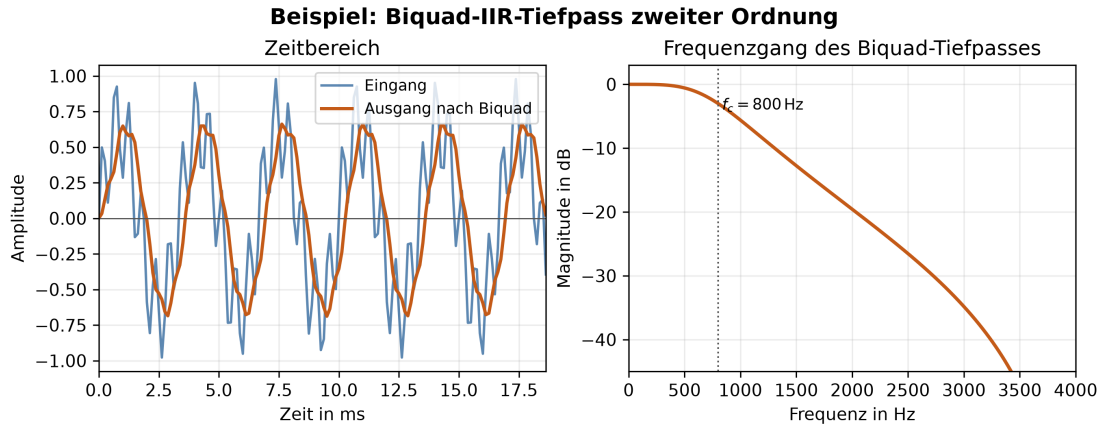


Abbildung 2.2: Beispiel eines Biquad-IIR-Tiefpasses zweiter Ordnung mit einer Grenzfrequenz von $f_c = 800$ Hz. Der Frequenzgang zeigt die Dämpfung hoher Frequenzanteile. Darstellung nach (Smith, 2007).

Ausgangswerte als Zustände speichern und bei jedem Sample aktualisieren. Für den Benchmark ist besonders relevant, dass diese Rückkopplung Datenabhängigkeiten erzeugt und die Vektorisierbarkeit gegenüber rein vorwärtsgerichteten Operationen einschränken kann. Entsprechend wird zwischen vektorisierbaren und skalaren Ausdrücken unterschieden; rekursive Filter bilden dabei einen wichtigen Sonderfall (Scaringella et al., 2003).

2.2.3 FIR-Filter und Faltung

Ein Finite-Impulse-Response-Filter (FIR-Filter) besitzt keine Rückkopplung. Jedes Ausgangssample entsteht aus einer gewichteten Summe aktueller und vergangener Eingangssamples. Für eine Filterlänge L gilt

$$y[n] = \sum_{k=0}^{L-1} b_k x[n-k]. \quad (2.12)$$

Diese Differenzengleichung entspricht der direkten Faltungsimplementierung eines FIR-Filters (Smith, 2007). Abbildung 2.3 zeigt einen FIR-Tiefpass mit 31 Filterkoeffizienten (Taps) und einer Grenzfrequenz von 800 Hz. Das synthetische Eingangssignal enthält einen niederfrequenten Anteil bei 300 Hz sowie einen höherfrequenten Anteil bei 1800 Hz; der FIR-Filter dämpft den höherfrequenten Anteil. Im Unterschied zum Biquad

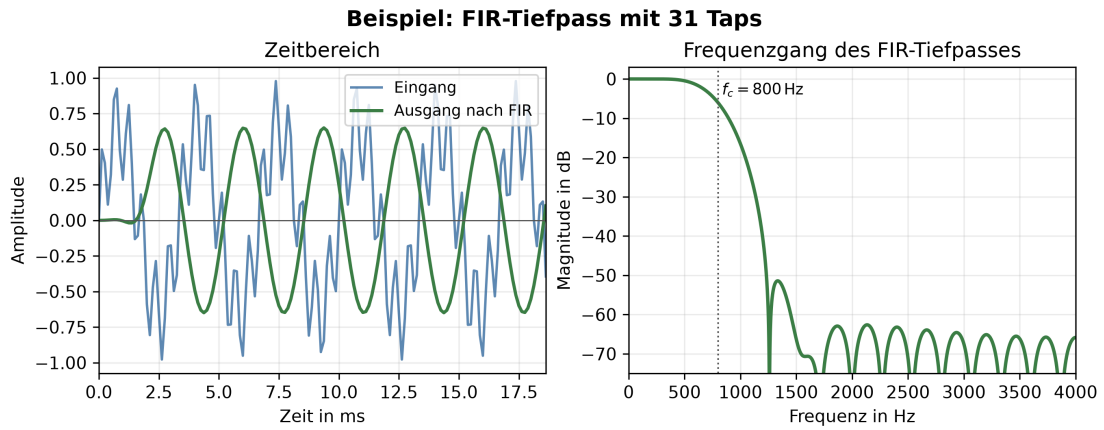


Abbildung 2.3: Beispiel eines FIR-Tiefpasses mit 31 Filterkoeffizienten und einer Grenzfrequenz von $f_c = 800$ Hz. Der Filter arbeitet ohne Rückkopplung und dämpft hohe Frequenzanteile. Darstellung nach (Smith, 2007).

hängt $y[n]$ nicht von früheren Ausgangswerten ab. Der Aufwand wächst mit der Zahl der Taps. Pro Ausgangssample beträgt die asymptotische Komplexität daher

$$O(L). \quad (2.13)$$

Die lineare Abhängigkeit von der Filterlänge ergibt sich aus der direkten Auswertung der Faltungssumme (Smith, 2007). Die Filterlänge ist für die Evaluation wesentlich: Sie erhöht sowohl die Zahl der Operationen als auch die Größe des benötigten Zustands. Nichtrekursive Filterstrukturen können vollständig vektorisiert werden, sofern keine weiteren Datenabhängigkeiten entgegenstehen (Scaringella et al., 2003).

2.3 Frequenzanalyse mit diskreter Fourier-Transformation (DFT) und Fast-Fourier-Transformation (FFT)

2.3.1 Zeitbereich, Frequenzbereich und DFT

Im Zeitbereich beschreibt ein digitales Signal seinen Amplitudenverlauf über dem Sample-Index. Im Frequenzbereich wird derselbe Signalabschnitt durch seine spektralen Komponenten dargestellt. Die diskrete Fourier-Transformation (DFT) ordnet einer endlichen Folge $x[n]$ der Länge N komplexe Koeffizienten $X[k]$ zu:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1. \quad (2.14)$$

Die Bedeutung der Indizes n , k und N folgt unmittelbar aus der DFT-Definition (Tan & Jiang, 2018, S. 97). Der Betrag eines Koeffizienten beschreibt die Stärke der jeweiligen Frequenzkomponente, während sein Argument die zugehörige Phase enthält. Der Abstand zwischen benachbarten Frequenz-Bins beträgt

$$\Delta f = \frac{f_s}{N}. \quad (2.15)$$

Abbildung 2.4 vergleicht Zeitbereich und Frequenzbereich anhand eines Einzeltons und eines Mischsignals. Diese Frequenzauflösung ergibt sich aus Abtastfrequenz und Transformationslänge (Tan & Jiang, 2018, S. 101, 103). Eine größere Transformationslänge N verbessert folglich die Frequenzauflösung, erhöht jedoch den Rechenaufwand.

2.3.2 Fast-Fourier-Transformation

Die Fast-Fourier-Transformation (FFT) berechnet dieselben DFT-Koeffizienten mit einer effizienteren Zerlegung des Problems. Während eine direkte DFT eine Komplexität von

$$O(N^2) \quad (2.16)$$

aufweist, reduziert eine radix-2-FFT den Aufwand auf

$$O(N \log_2 N). \quad (2.17)$$

Diese Beschleunigung ergibt sich aus der Rekursion der N -Punkt-DFT in kleinere Teiltransformationen; die asymptotischen Laufzeiten werden als Konsequenz dieser Zerlegung begründet (Tan & Jiang, 2018, S. 127–134). Die FFT-Größe stellt deshalb im Benchmark einen relevanten Skalierungsparameter dar: Sie verändert sowohl den theoretischen Rechenaufwand als auch die praktische Speicher- und Cache-Nutzung. Dieses Kapitel behandelt ausschließlich die Vorwärtstransformation, weil die Forschungsfrage einzelne FFT-Bausteine vergleicht; eine vollständige Short-Time-Fourier-Transform-(STFT)-Pipeline oder eine Overlap-Add-Pipeline ist nicht Teil dieses Grundlagenkapitels.

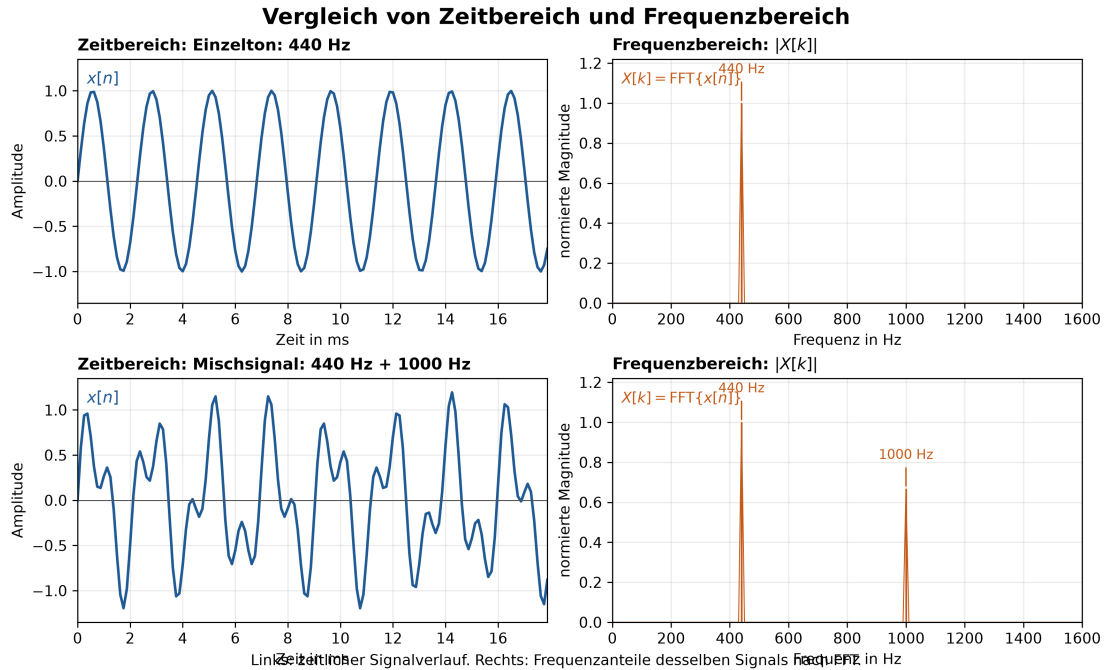


Abbildung 2.4: Vergleich von Zeitbereich und Frequenzbereich. Oben ist ein 440-Hz-Einzelton dargestellt, der im Spektrum einen dominanten Frequenzanteil bei 440 Hz erzeugt. Unten ist ein Mischsignal aus 440 Hz und 1000 Hz dargestellt. Beide Anteile erscheinen im Frequenzbereich als separate Peaks. Darstellung auf Grundlage der DFT/FFT-Spektralanalyse (Tan & Jiang, 2018, S. 102).

2.4 Performanzrelevante Implementierungsaspekte

2.4.1 Compileroptimierung und automatische Vektorisierung

Die Laufzeit eines DSP-Kerns wird nicht allein durch seinen Algorithmus bestimmt, sondern auch durch die Codegenerierung und die Optimierungen des Compilers. Bei automatischer Vektorisierung versucht der Compiler, mehrere unabhängige skalare Operationen durch SIMD-Instruktionen zusammenzufassen. Für Faust existiert eine Analyse, die vektorisierbare Ausdrücke von Ausdrücken mit Datenabhängigkeiten trennt (Scaringella et al., 2003). Diese Unterscheidung erklärt, warum Verstärkungsstufen und lange FIR-Strukturen andere Optimierungsmöglichkeiten besitzen können als rekursive Biquad-Filter. Die Arbeit vergleicht beide Implementierungen unter identischen Compiler- und Optimierungsbedingungen. Explizite SIMD-Intrinsics werden nicht eingesetzt, damit die Wirkung der automatischen Optimierung für Faust und C++ beobachtbar bleibt. Die konkrete Wahl von Compiler, Flags und Zielplattform wird erst im Versuchsdesign dokumentiert.

2.4.2 Messgrößen

Die zentrale Messgröße ist die Prozessorzeit pro Audioblock. Ergänzend werden Speicherverbrauch und Binärgröße erfasst, weil sie die praktische Einsetzbarkeit einer Implementierung neben der reinen Laufzeit beeinflussen. Codelänge und Implementierungskomplexität ergänzen den quantitativen Vergleich um Aspekte der Entwicklererfahrung. Die genaue Operationalisierung dieser Größen, die Zahl der Wiederholungen sowie die statistische Auswertung gehören in Kapitel 4.

2.5 Zusammenfassung

Die Grundlagen beschreiben die für die Benchmarks benötigte Verarbeitungskette von digitalen Samples über Gain-, IIR- und FIR-Operationen bis zur FFT. Blockgröße, Filterlänge und FFT-Größe bestimmen dabei den Rechenaufwand und bilden die zentralen Skalierungsparameter der Evaluation. Die folgenden Kapitel erläutern anschließend Faust und das konkrete Versuchsdesign, ohne die Grundlagen um anwendungsfremde Verfahren der Sprachrauschunterdrückung zu erweitern.

Kapitel 3

Faust: Functional Audio Stream

3.1 Einführung

FAUST (Functional Audio Stream) ist eine domänenspezifische Programmiersprache für die Audio-Signalverarbeitung. Sie ist als effiziente Ergänzung zu C/C++ für Signalverarbeitungsbibliotheken, Audio-Plug-ins und eigenständige Anwendungen konzipiert (Orlarey et al., 2009).

Anders als visuelle Sprachen wie Max oder Pure Data ist FAUST eine kompilierte Sprache. Der FAUST-Compiler übersetzt FAUST-Programme direkt in C++-Code. Dieser Code ist lauffähig, selbstständig und benötigt keine externe Laufzeitumgebung. Er arbeitet deterministisch und mit konstantem Speicherverbrauch. Vgl. Orlarey et al., 2009, S. 2

Für die vorliegende Arbeit ist FAUST relevant, weil die Sprache Algorithmen der digitalen Signalverarbeitung (Digital Signal Processing, DSP) als funktionale Blockdiagramme beschreibt. Der Compiler erzeugt daraus C++-Code, der in dieselbe Benchmark-Infrastruktur wie eine manuell implementierte C++-Referenz eingebunden werden kann (Orlarey et al., 2009, S. 2–3). Der Vergleich betrifft die vom FAUST-Compiler abgeleitete und eine manuelle Implementierung desselben DSP-Algorithmus.

Die deklarative Blockdiagrammbeschreibung und die Trennung von Spezifikation und Implementierung können Codelänge und Implementierungskomplexität beeinflussen. Compileranalyse und Codegenerierung können sich zudem auf Laufzeit, Speicherverbrauch und Binärgröße auswirken. Ergebnisse aus FAUST-STK deuten für bestimmte physikalische Modelle auf kompaktere FAUST-Quelltexte hin (Michon & Smith, 2011, S. 5–6). Diese Ergebnisse sind nicht ohne Weiteres auf Gain-Stufen, Biquad- und FIR-Filter oder Fast-Fourier-Transformationen (FFTs) übertragbar. Die Überprüfung für diese DSP-Bausteine ist daher Gegenstand der vorliegenden Untersuchung.

Die Besonderheit von FAUST liegt in der formalen Semantik: Jedes Programm beschreibt eine mathematische Funktion, die Eingangssignale in Ausgangssignale transformiert. Der Compiler übersetzt nicht den Programmtext, sondern die dahinterstehende Funktion. Dadurch können zwei syntaktisch unterschiedliche, aber semantisch äquivalente Programme denselben Code erzeugen. (Orlarey et al., 2009)

Programm 3.1: Minimale FAUST-Signalprozessoren

```

1  process = 0;          // erzeugt Stille
2  process = _;          // kopiert Eingang auf Ausgang
3  process = +;          // addiert zwei Kanäle (Stereo zu Mono)
4

```

3.2 Programmmodell

FAUST kombiniert funktionale Programmierung mit Blockdiagrammen. Digitale Signale werden als diskrete Funktionen der Zeit modelliert. Signalprozessoren verarbeiten diese Signale, und Kompositionsoperatoren verbinden mehrere Signalprozessoren zu größeren Blockdiagrammen (Orlarey et al., 2009, S. 3–4).

Ein FAUST-Programm beschreibt keinen Klang, sondern einen Signalprozessor, also eine Maschine mit Eingängen und Ausgängen. Das Schlüsselwort **process** definiert diesen Signalprozessor 3.1 zeigt drei minimale Definitionen.

FAUST-Primitive funktionieren wie C-Funktionen, aber auf Signalen. Beispielsweise berechnet **sin** den Sinus jedes Samples eines Eingangssignals. Der Verzögerungsoperator **@** bildet ein Signal mit einer zeitvariablen Verzögerung ab (Orlarey et al., 2009).

3.3 Syntax: Blockdiagramm-Komposition

Tabelle 3.1 fasst die fünf Kompositionsoperatoren der FAUST-Blockdiagrammalgebra zusammen (Orlarey et al., 2009, S. 5).

Tabelle 3.1: Kompositionsoperatoren der Faust Block-Diagramm-Algebra

Operator	Bezeichnung	Beschreibung
$f \sim g$	Rekursive Komposition	Feedback-Schleife mit 1-Sample-Delay
f, g	Parallele Komposition	Zwei Signalprozessoren nebeneinander
$f : g$	Sequenzielle Komposition	Ausgang von f mit Eingang von g verbinden
$f < : g$	Split	Ein Ausgang auf mehrere Eingänge verteilen
$f > : g$	Merge	Mehrere Ausgänge auf einen Eingang zusammenführen

Beispiel für sequenzielle Komposition: **process = + : abs;** verbindet die Ausgabe einer Addition mit einem Absolutwert. (Orlarey et al., 2009, S. 4)

Beispiel für rekursive Komposition: **process = + _;** erzeugt einen Integrator mit einer impliziten Verzögerung um ein Sample $Y(t) = X(t) + Y(t - 1)$. (Orlarey et al., 2009, S. 4)

Programm 3.2 zeigt einen Rauschgenerator mit rekursiver Pseudozufallszahlenerzeugung und einem Regler für die Signalamplitude (Orlarey et al., 2009, S. 5)

Die rekursive Definition erzeugt eine Pseudozufallsfolge. Die nachfolgenden Definitionen skalieren sie in den Bereich von minus eins bis eins und verknüpfen sie mit einem interaktiven Lautstärkeregler, um den resultierenden Signal zu steuern (Orlarey et al.,

Programm 3.2: Rauschgenerator mit Lautstärkeregler in FAUST

```
1  random = +(12345) ~ *(1103515245);
2  noise = random / 2147483647.0;
3  process = noise * vslider("noise[style:knob]", 0, 0, 100, 0.1) / 100;
4
```

2009, S. 6).

3.4 Semantik

FAUST besitzt eine denotationale Semantik. Ein Programm beschreibt nicht eine Folge imperativer Einzelschritte, sondern die mathematische Bedeutung eines Signalprozessors. Digitale Signale werden dabei als diskrete Funktionen der Zeit aufgefasst. Ein Signalprozessor ist entsprechend eine Funktion, die Eingangs- in Ausgangssignale überführt (Orlarey et al., 2009, S. 2–4).

Diese Sichtweise trennt die Spezifikation eines DSP-Algorithmus von dessen konkreter Implementierung. Die FAUST-Quellbeschreibung legt fest, welche Signaltransformation berechnet werden soll. Der Compiler entscheidet anschließend, wie sie effizient in C++ umgesetzt wird. Semantisch äquivalente Blockdiagramme können nach der Normalisierung dieselbe Implementierung erzeugen (Orlarey et al., 2009, S. 3, S. 12).

Für die vorliegende Arbeit ist diese Trennung wesentlich. Die FAUST-Variante und die manuelle C++-Referenz können unterschiedlich strukturiert sein. Beide müssen jedoch denselben Algorithmus und dieselbe Ein-/Ausgabesemantik realisieren. Der Vergleich bewertet daher die Qualität der abgeleiteten Implementierung und nicht lediglich Unterschiede in der Oberflächensyntax.

3.5 Compiler-Architektur

Der FAUST-Compiler verarbeitet eine Quellbeschreibung in mehreren aufeinander aufbauenden Analysephasen. Diese Phasen trennen die sprachliche Auswertung, die Ermittlung der mathematischen Bedeutung und die Optimierungsvorbereitung von der anschließenden C++-Codegenerierung (Orlarey et al., 2009, S. 10–13).

1. **Einlesen und Parsen der Quelldateien.** Der Compiler liest die Hauptdatei sowie über `import`-Direktiven eingebundene Dateien rekursiv ein. Das Ergebnis ist eine Menge benannter Definitionen. (Orlarey et al., 2009, S. 10–11).
2. **Auswertung der Blockdiagrammdefinitionen.** Algorithmische Definitionen werden ausgewertet und in eine explizite Blockdiagrammbeschreibung in Normalform überführt. Für die weitere Analyse liegen damit die primitiven Signaloperationen und ihre Verschaltung unmittelbar vor (Orlarey et al., 2009, S. 11–12).
3. **Symbolische Semantikanalyse und Normalisierung.** Der Compiler propagiert symbolische Eingangssignale durch das Blockdiagramm. Daraus leitet er für jedes Ausgangssignal eine mathematische Gleichung ab. Anschließend normalisiert

Programm 3.3: Semantisch äquivalente FAUST-Beschreibungen

```

1    process = /(2) : @ (10);
2    process = *(2) : @ (7) : /(4) : @ (3);
3

```

er diese Gleichungen, sodass semantisch äquivalente Blockdiagramme dieselbe Repräsentation erhalten. Aufeinanderfolgende Verzögerungen können zusammengefasst werden. Konstante Faktoren können so angeordnet werden, dass Verzögerungsleitungen wiederverwendbar sind (Orlarey et al., 2009, S. 12).

Programm 3.3 zeigt zwei syntaktisch unterschiedliche Programme mit derselben Signalgleichung.

Beide Beschreibungen ergeben nach der Normalisierung dieselbe Ausgangsgleichung. Sie können deshalb auf dieselbe optimierte Implementierung abgebildet werden (Orlarey et al., 2009, S. 12).

4. **Typ- und Bereichsanalyse.** Für jede resultierende Signalgleichung ermittelt der Compiler den numerischen Datentyp, den möglichen Wertebereich, den Berechnungszeitpunkt, die Ausführungsgeschwindigkeit und die Parallelisierbarkeit. Konstante Signale können einmalig berechnet werden. Bedienelemente können blockweise und Audiosignale sampleweise verarbeitet werden. Diese Unterscheidungen sind Implementierungsdetails; aus Sicht der FAUST-Programmierung bleiben alle Werte reguläre Audiosignale (Orlarey et al., 2009, S. 12–13).
5. **Vorkommenanalyse.** Der Compiler untersucht, in welchen Kontexten Teilausdrücke auftreten und ob sie gemeinsam verwendet werden. Teure gemeinsame Teilausdrücke können in temporären Variablen zwischengespeichert werden. Dies gilt auch für Ausdrücke, die zwischen unterschiedlichen Geschwindigkeitskontexten benötigt werden. Nicht jeder mehrfach auftretende Teilausdruck wird jedoch automatisch ausgelagert (Orlarey et al., 2009, S. 13).

Erst nach diesen Phasen beginnt die Codegenerierung. Der Compiler übersetzt somit nicht die ursprüngliche Schreibweise des Programms, sondern eine analysierte und normalisierte Repräsentation seiner Signalverarbeitung (Orlarey et al., 2009, S. 13).

Erst nach diesen Phasen beginnt die Codegenerierung. Der Compiler übersetzt somit nicht die ursprüngliche Schreibweise des Programms, sondern eine analysierte und normalisierte Repräsentation seiner Signalverarbeitung (Orlarey et al., 2009, S. 13).

3.6 Codegenerierung

Auf Basis der analysierten Signalgleichungen erzeugt der FAUST-Compiler C++-Code. Die skalare Variante bildet die Grundlage. Der Vektor-Modus strukturiert diesen Code für die automatische Vektorisierung um. Der Parallel-Modus ergänzt den Vektor-Code um Direktiven für die nebenläufige Ausführung (Orlarey et al., 2009, S. 13–21).

Programm 3.4: Zwischenspeicherung eines gemeinsam verwendeten Teilausdrucks

```
1 // Stereo zu Mono: kein Caching nötig
2 process = +;
3 → output0[i] = input0[i] + input1[i];
4
5 // Split: Ergebnis wird zwischengespeichert
6 process = + <: ,;
7 → float fTemp0 = input0[i] + input1[i];
8 output0[i] = fTemp0;
9 output1[i] = fTemp0;
10
```

3.6.1 Skalarer Modus

Im skalaren Modus berechnet eine Schleife die Ausgänge sampleweise. Für jede Addition $E_1 + E_2$ wird der Code von E_1 und E_2 generiert und mit einem $+$ verbunden. Wird das Ergebnis mehrfach verwendet, wird eine Zwischenspeicher-Variable angelegt, um redundante Berechnungen zu vermeiden (Orlarey et al., 2009, S. 13–14).

Programm 3.4 veranschaulicht den Unterschied zwischen einer direkten Berechnung und einer zwischengespeicherten Berechnung.

3.6.2 Vektor-Modus

Der Vektor-Modus erzeugt mehrere einfachere Schleifen, die über Vektoren kommunizieren. Dadurch erhält der nachgelagerte C++-Compiler eine Codeform, die die automatische Vektorisierung erleichtert. Gemeinsam verwendete, rekursive oder für Verzögerungsleitungen benötigte Ausdrücke können in eigenen Schleifen berechnet werden. Das Ergebnis ist ein gerichteter azyklischer Graph aus Berechnungsschleifen (Orlarey et al., 2009, S. 15–17).

Die automatische Vektorisierung unterscheidet vektorisierbare von nicht vektorisierbaren Ausdrücken anhand von Typ- und Kontextinformationen. Die dadurch erzielbare Beschleunigung hängt von Algorithmus, Compiler und Zielarchitektur ab und darf daher nicht als allgemeiner Leistungsgewinn interpretiert werden (Scaringella et al., 2003).

3.6.3 Parallel-Modus

Der Parallel-Modus baut auf dem Vektor-Modus auf und ergänzt den erzeugten C++-Code um Open Multi-Processing (OpenMP)-Direktiven. Unabhängige Schleifen desselben Ausführungsniveaus können parallel ausgeführt werden. Rekursive Abhängigkeiten begrenzen diese Parallelisierung (Orlarey et al., 2009, S. 17–21).

Spätere Arbeiten untersuchen zusätzlich dynamische Scheduling-Verfahren für den Schleifengraphen. Diese Verfahren ändern jedoch nichts daran, dass nutzbarer Parallelismus und der Aufwand der Synchronisierung von der Struktur des Signalprozessors abhängen (Letz et al., 2010).

3.6.4 Generierte Code

Der erzeugte C++-Code wird als Klasse ausgegeben. Zu den zentralen Methoden zählen die Abfrage der Ein- und Ausgänge, die Initialisierung, die Beschreibung der Benutzeroberfläche und die eigentliche Signalverarbeitung (Orlarey et al., 2009, S. 6–7).

- `getNumInputs()` / `getNumOutputs()` - Anzahl Ein-/Ausgänge
- `init(int samplingFreq)` - Initialisierung
- `buildUserInterface(UI*)` - GUI-Beschreibung
- `compute(int count, float** input, float** output)` - Signalverarbeitungsmethode

3.7 Backends und Architekturdateien

Architekturdateien trennen die Signalverarbeitungslogik von der Einbettung in ein Audio- oder Benutzeroberflächensystem. Sie umschließen die generierte C++-Klasse. Dadurch kann dieselbe FAUST-Quellbeschreibung für verschiedene Anwendungen und Plug-in-Formate verwendet werden (Orlarey et al., 2009, S. 8).

Die in diesem Abschnitt dargestellte Architekturauswahl stammt aus einer älteren FAUST-Version. Für aktuelle Zielsprachen, Zielplattformen und Architekturdateien ist deshalb die offizielle FAUST-Dokumentation maßgeblich (GRAME - Centre National de Création Musicale, 2025).

3.8 Performance

Die in der Grundquelle dargestellten Benchmarks vergleichen mehrere Testprogramme auf bestimmten Hardware- und Compilerkonfigurationen. Sie zeigen, dass der Nutzen von Vektorisierung und Parallelisierung von der Struktur des Signalprozessors, dem verfügbaren Parallelismus und der Speicherbandbreite abhängt (Orlarey et al., 2009, S. 22–28).

Die Messungen wurden mit einer historischen FAUST-Version sowie den damals verfügbaren Compilern und Testsystemen durchgeführt. Sie dienen deshalb der Einordnung der Compilerstrategien, erlauben jedoch keine Vorhersage für die in dieser Arbeit verwendete Hardware, Compilerkonfiguration oder Benchmark-Infrastruktur (Orlarey et al., 2009, S. 22–23).

- **Speicherbandbreitenbegrenzte Programme.** Im Benchmark `copy1` waren die skalare und die vektorisierte Variante ähnlich schnell. Die parallele Variante schnitt wegen der geteilten Speicherbandbreite schlechter ab (Orlarey et al., 2009, S. 22–23).
- **Programme mit begrenztem Parallelismus.** Für die Freeverb-Implementierung berichten die Autoren bei Verwendung des Intel-C++-Compilers auf zwei der drei Testsysteme moderate Zugewinne durch Vektorisierung und Parallelisierung (Orlarey et al., 2009, S. 24).

- **Programme mit hohem Parallelismus.** Für `karplus32` und `rms8` wurden mit dem Intel-C++-Compiler deutliche Leistungssteigerungen durch Vektorisierung und Parallelisierung beobachtet. Die Höhe des Gewinns blieb jedoch von Hardware und Speicherbandbreite abhängig (Orlarey et al., 2009, S. 24, S. 27–28)

3.9 Zusammenfassung

FAUST ist eine kompilierte Sprache für Audio-Signalverarbeitung mit formaler Semantik. Die Sprache trennt die Beschreibung eines Signalprozessors von dessen Implementierung und ermöglicht dem Compiler, daraus eigenständigen C++-Code zu erzeugen (Orlarey et al., 2009). Für die vorliegende Arbeit ist insbesondere relevant, dass dieselben DSP-Algorithmen sowohl als FAUST-Programm als auch als manuelle C++-Referenz implementiert und unter einer gemeinsamen Benchmark-Infrastruktur verglichen werden können.

Kapitel 4

Versuchsdesign und Benchmark-Methodik

4.1 Definition der untersuchten DSP-Bausteine

warum folgende Bausteine; Code;

4.2 Benchmark-Tools und Messgrößen

Wie oft etwas durchgelaufen wurde und womit;

4.3 Hardwareumgebung

WSL Umgebung und Entwicklungsumgebung

4.4 Compilerkonfigurationen

Flag-Matrix und ihre Begründungen

4.5 Testdatengenerierung

Mit welchen Testdaten gearbeitet wurde

Kapitel 5

Implementierung der Benchmarks

5.1 Umsetzung der FAUST- und C++-Code

Codebeispiele

5.2 Gemeinsame Test-Schnittstelle

5.3 Build-Systeme

5.4 Sicherstellung einer fairen Vergleichsbasis

Gleiche flags; Gleiche Inputs;

5.5 Verifikation der Ergebnisse

Zeigen dass beide Codes die selben Ergebnisse liefern.

Kapitel 6

Evaluation der Messergebnisse

6.1 Mikrobenchmark-Ergebnisse

je Baustein: CPU-Zeit/Block; Durchsatz; Speicherverbrauch, Binaergröße Grafiken

6.2 Einfluss von Compileroptimierungen und Codegenerierung

-O0 -> -O3 und -fastmath

Vektorisierung wurde nicht berücksichtigt; Gibt schon eine studie

6.3 Numerische Genauigkeit

6.4 Codelänge und Implementierungskomplexität

6.5 Zusammenfassung der Messergebnisse

Kapitel 7

Diskussion

- 7.1 Interpretation der Ergebnisse im Hinblick auf die Forschungsfrage
- 7.2 Limitationen des Versuchsaufbaus
- 7.3 Bedrohung der Validität

Kapitel 8

Zusammenfassung und Ausblick

8.1 Zusammenfassung der zentralen Ergebnisse

8.2 Ausblick auf künftige Forschungsarbeiten

Quellenverzeichnis

Literatur

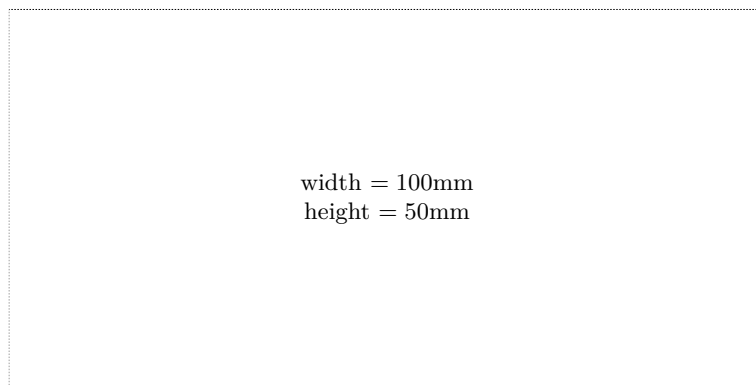
- Ifeachor, E. C., & Jervis, B. W. (2002). *Digital signal processing: a practical approach*. Pearson Education. (Siehe S. 1).
- Letz, S., Orlarey, Y., & Foer, D. (2010). Work Stealing Scheduler for Automatic Parallelization in Faust. In LAC (Hrsg.), *Proceedings of Linux Audio Conference*. <https://hal.science/hal-02158924> (siehe S. 2, 17).
- Levine, N., & Skolnick, D. (1997). DSP 101 Part 3: Implement Algorithms on a Hardware Platform. *Analog Dialogue*, 31(3). Verfügbar 4. August 2026 unter <https://www.analog.com/en/resources/analog-dialogue/articles/dsp-101-part-3.html> (siehe S. 1).
- Michon, R., & Smith, J. O. (2011). FAUST-STK: A Set of Linear and Nonlinear Physical Models for the FAUST Programming Language. *Proceedings of the 14th International Conference on Digital Audio Effects (DAFx-11)* (siehe S. 1, 2, 13).
- Orlarey, Y., Foer, D., & Letz, S. (2009). FAUST : an Efficient Functional Approach to DSP Programming. In E. D. FRANCE (Hrsg.), *NEW COMPUTATIONAL PARADIGMS FOR COMPUTER MUSIC* (S. 65–96). <https://hal.science/hal-02159014> (siehe S. 1, 2, 13–19).
- Rossing, T. (2007). *Springer Handbook of Acoustics*. Springer New York. https://books.google.at/books?id=4ktVwGe_dSMC (siehe S. 5).
- Scaringella, N., Orlarey, Y., Letz, S., & Foer, D. (2003). Automatic vectorization in Faust. In JIM (Hrsg.), *Actes des Journées d'Informatique Musicale JIM2003*. <https://hal.science/hal-02158949> (siehe S. 2, 8, 9, 11, 17).
- Smith, J. O. (2007). *Introduction to digital filters: with audio applications* (Bd. 2). Julius Smith. (Siehe S. 7–9).
- Tan, L., & Jiang, J. (2018). *Digital Signal Processing: Fundamentals and Applications*. Academic Press. <https://books.google.at/books?id=MxlxDwAAQBAJ> (siehe S. 5, 6, 10, 11).

Online-Quellen

- GRAME - Centre National de Création Musicale. (2025). *Faust Manual*. Verfügbar 20. Oktober 2025 unter <https://faustdoc.grame.fr/manual/introduction/> (siehe S. 1, 18).

Check Final Print Size

— Check final print size! —



— Remove this page after printing! —